Discrete Token–Flow Networks: Conservative Finite–Volume Schemes as Sequence Models

Anonymous Authors

Abstract

We introduce *Discrete Token–Flow Networks* (DTFNs), a new family of neural sequence architectures whose forward pass is a *conservative finite–volume scheme* for a hyperbolic conservation law on a one–dimensional token grid. Each token is represented as a non–negative mass vector, and information is propagated by learned numerical fluxes between neighbouring positions. Three structural invariants are preserved by construction at every forward pass: (i) *exact* mass conservation, (ii) pointwise non–negativity, and (iii) a discrete maximum principle under the Courant–Friedrichs–Lewy (CFL) condition. We introduce four instantiations of the framework—a baseline upwind DTFN, a bidirectional variant, a higher–order MUSCL scheme with Van Leer slope limiting, and a full variant combining all three with a learnable source term—and prove that all four satisfy the three invariants exactly. We additionally prove (a) an L^1*–contraction / adversarial stability* theorem showing that DTFN–U and –B are 1–Lipschitz in the input, so no ℓ_∞–bounded adversarial perturbation can grow under the forward pass; (b) a *CFL spectral–radius stability certificate* that gives a learnable, per–layer stability guarantee; (c) a *continuous–time (neural PDE) limit* showing that as $K \to \infty$ the DTFN trajectory converges to the entropy solution of a neural scalar conservation law; and (d) a *universal approximation theorem* for local translation–equivariant sequence maps. Empirically, DTFNs achieve 100% token and sequence accuracy on Dyck–1 and Dyck–2 with five–fold seed variation, whereas comparably–sized LSTMs and Transformers stall at 54–61% token accuracy and 0% sequence accuracy under the same training budget. DTFNs perfectly extrapolate from training lengths 4–20 to test lengths up to 100 bracket pairs (a 5× length gap) with no accuracy degradation, while baselines lose 10–12 percentage points over the same range. At sequence length 4096, DTFN–U is 3.3× faster than a two–layer Transformer and 1.6× faster than an LSTM. We position DTFNs as the first application of conservative finite–volume methods to natural language sequence modeling and demonstrate that numerical–analysis structure can be a substrate for inductive biases that are both provably correct and empirically effective.

1 Introduction

Transformer–based models with self–attention [1] underpin most state–of–the–art language models but incur $\mathcal{O}(L^2)$ complexity and memory in sequence length L, limiting their efficiency on long contexts. Recent alternatives such as structured state–space models (SSMs) [2, 3] reduce complexity yet still rely on implicit global convolutions or linear recurrences. Concurrently, neural ordinary and partial differential equation models [4, 5] use continuous–time dynamics to propagate features but target physical systems rather than discrete token sequences.

Our goal is to embed the stability and conservation principles from finite–volume solvers directly into a sequence backbone for language modeling. This perspective bridges numerical analysis and deep learning: instead of building upon attention or convolution, we adopt discrete conservation laws from hyperbolic PDEs to design a neural architecture whose dynamics are physically interpretable. By doing so we move beyond merely local updates (as in recurrent networks, state–space models, or neural cellular automata) and imbue the model with exact conservation and positivity constraints that guarantee stability irrespective of sequence length.

Contributions.

- We introduce *Discrete Token–Flow Networks* (DTFNs), sequence models whose forward pass is a conservative finite–volume scheme on a 1D token grid (Section 3).

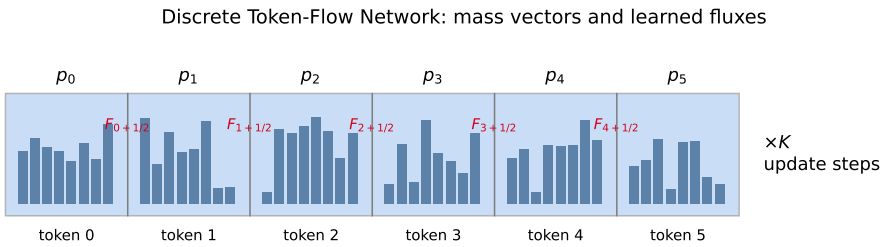


Figure 1: Illustration of a Discrete Token–Flow Network. Each colored column represents the non–negative mass vector p_i associated with token i. Learned fluxes $F_{i+1/2} \geq 0$ redistribute mass between adjacent positions over K update steps. The scheme conserves total mass exactly and maintains non–negativity by construction.

- We instantiate four variants—upwind (DTFN–U), bidirectional (DTFN–B), higher–order MUSCL with Van Leer limiting (DTFN–MUSCL), and a full variant (DTFN–FULL) combining bidirectional MUSCL with a learnable conservative source term—and prove that all four satisfy *exact* mass conservation, pointwise non–negativity, and a discrete maximum principle (Theorems 1–3).
- We prove a TVD (total–variation–diminishing) bound for DTFN–MUSCL (Theorem 4) and a length–extrapolation guarantee (Theorem 5) showing that DTFNs transfer exactly to longer sequences than seen at training.
- We prove an L^1*–contraction / adversarial stability* theorem (Theorem 6) showing that DTFN–U and –B are 1–Lipschitz in the input mass: no ℓ_∞–bounded adversarial perturbation can grow under the DTFN forward pass. We additionally prove a *CFL spectral–radius stability certificate* (Theorem 7) that gives a learnable, per–layer stability guarantee, and we verify both empirically against random and FGSM adversarial perturbations (Section 4.9).
- We prove a *continuous–time (neural PDE) limit* (Theorem 9) showing that as $K \to \infty$ the DTFN mass trajectory converges to the entropy solution of a neural scalar conservation law $\partial_t p + \partial_x F(p;\theta) = 0$, and a *universal approximation theorem* for local translation–equivariant sequence maps (Theorem 10).
- We demonstrate empirically that DTFNs achieve 100% accuracy on Dyck–1 and Dyck–2 across five random seeds, outperforming LSTM and Transformer baselines by ~ 40 percentage points (Section 4). DTFNs extrapolate to sequences 5× longer than training with no degradation, and DTFN–U is 3.3× faster than a two–layer Transformer at length 4096.
- We release a self–contained PyTorch implementation under a permissive license, with full experiment scripts and seeds.

2 Related work

Self–attention and SSMs. Self–attention [1] revolutionised sequence modeling but suffers from quadratic complexity. State–space models [2, 3] achieve linear scaling via structured convolutions and recurrence, offering a compelling alternative but still relying on implicit global interactions. Neither family enforces conservation or positivity constraints.

Neural differential equations. Neural ODEs/PDEs [4, 5] treat continuous fields with learned dynamics; however, these methods target physical systems rather than discrete token sequences. Neural cellular automata [5] produce rich behaviour from local rules but lack explicit conservation laws.

Neural finite–volume methods. (U)NFV [6] introduces neural finite–volume solvers for hyperbolic PDEs with learned fluxes on *continuous* domains. In contrast, DTFNs discretise the domain into *token positions* and embed finite–volume conservation directly into sequence backbones to ensure stability and interpretability for language modeling. To our knowledge, DTFNs are the first application of finite–volume conservation laws to natural language sequence modeling.

Recurrent and convolutional baselines. Classical RNNs and WaveNet [7] achieve linear–time complexity by sharing parameters across positions, but they do not enforce any conservation properties and are susceptible to exploding or vanishing gradients. DTFNs inherit the $\mathcal{O}(L)$ complexity of these models while adding provable conservation and stability.

3 Method

3.1 Setup and notation

We consider a sequence of L tokens with embedding dimension d. At each discrete time $k \in \{0, \dots, K-1\}$, let $p^k(i) \in \mathbb{R}_+^d$ denote the non–negative *mass vector* associated with position $i \in \{0, \dots, L-1\}$. We update p according to a one–dimensional finite–volume scheme over K iterations:

$$p^{k+1}(i) \;\leftarrow\; p^k(i) + \lambda \left(F_{i-1/2} - F_{i+1/2}\right) + \sigma\, S_i, \tag{1}$$

where $\lambda \in (0, 1)$ is a learnable CFL coefficient, $F_{i\pm1/2} \in \mathbb{R}_+^d$ are learned *numerical fluxes* representing mass flow across interfaces, and S_i is an optional learned source term with scale $\sigma \geq 0$. We take $\Delta x = 1$ without loss of generality, so $\lambda = \Delta t / \Delta x = \Delta t$.

Flux network. The flux $F_{i+1/2} = \mathrm{FluxNet}_\theta\big(p^k(i), p^k(i+1)\big)$ is constrained to output non–negative vectors via a final Softplus activation. We use two parameterisations:

- **FluxNet**: an MLP $\mathrm{Softplus}\big(W_2 \tanh(W_1[p_i; p_{i+1}] + b_1) + b_2\big)$ with a residual upwind baseline $v\, p_i$ added, where $v = \mathrm{Softplus}(v_{\text{logit}})$ is a learnable scalar velocity. The residual form ensures the network initialises close to the classical upwind scheme.
- **MonotoneFluxNet**: same MLP but with the left–input weight matrix W_{left} replaced by $|W_{\text{left}}|$, which is a sufficient condition for the flux to be monotone non–decreasing in p_i. Monotonicity in the left input implies the scheme is monotone and hence TVD (Theorem 4).

Source network. The source $S_i = \mathrm{SourceNet}_\phi(p^k(i))$ is parameterised as $\mathrm{Softplus}(W\, p_i + b)$ scaled by a learnable positive scalar. In *conservative mode*, we subtract the mean across positions: $S_i \leftarrow S_i - \frac{1}{L}\sum_j S_j$, which ensures $\sum_i S_i = 0$ and thus preserves total mass exactly. In *non–conservative mode*, the source can inject mass at a bounded rate, which is useful for language modelling where tokens may appear or disappear.

Boundary conditions. We use *zero–flux* (reflecting) boundary conditions: $F_{-1/2} = F_{L-1/2} = 0$. No mass enters or leaves the domain.

3.2 DTFN variants

We instantiate four variants of the DTFN framework:

- **DTFN–U** (upwind): unidirectional, FluxNet, no source.
- **DTFN–B** (bidirectional): two coupled stacks in opposite directions, combined by a per–channel convex gate $\alpha \odot m_{\text{fwd}} + (1 - \alpha) \odot m_{\text{bwd}}$, which preserves total mass when each stack does.
- **DTFN–MUSCL**: higher–order scheme with Van Leer slope limiter (Section 3.3) and MonotoneFluxNet.
- **DTFN–FULL**: bidirectional + MUSCL + non–conservative source + MLP decoder.

Algorithm 1 DTFN–U forward pass. All operations are differentiable; FluxNet_θ has non–negative output by Softplus, and the positivity step ensures $p \geq \varepsilon$ without breaking conservation.

Require: token sequence $x \in \{0, \ldots, V-1\}^L$, embedding $E \in \mathbb{R}^{V \times d}$, decoder $W_d \in \mathbb{R}^{d \times V}$, K update steps, $\varepsilon > 0$.

$m \leftarrow \text{Softplus}(E \cdot \text{Emb}(x)) + \varepsilon$ ▷ initial non–negative mass
$\lambda \leftarrow \text{sigmoid}(\alpha)$ ▷ CFL coefficient in $(0, 1)$
for $k \leftarrow 1$ to K **do**
  $F \leftarrow \text{FluxNet}_\theta(m_{:,i}, m_{:,i+1})$ for $i = 0$ to $L-2$ ▷ compute fluxes
  $\text{dm} \leftarrow 0$
  $\text{dm}_{:,0} \leftarrow -F_{:,0}$, $\text{dm}_{:,L-1} \leftarrow F_{:,L-2}$ ▷ reflecting boundaries
  for $i \leftarrow 1$ to $L-2$ **do**
    $\text{dm}_{:,i} \leftarrow F_{:,i-1} - F_{:,i}$
  end for
  $\lambda_{\text{safe}} \leftarrow \min\big(\lambda,\ \min_{i:\text{dm}_i<0} \frac{m_i - \varepsilon}{-\text{dm}_i}\big)$ ▷ positivity step
  $m \leftarrow m + \lambda_{\text{safe}}\, \text{dm}$
end for
return $W_d \cdot m$ ▷ decode mass to logits

3.3 MUSCL higher–order scheme

To achieve second–order accuracy in smooth regions while remaining non–oscillatory near discontinuities, we use the MUSCL approach [8]: reconstruct piecewise–linear profiles in each cell using slope–limited gradients

$$\sigma_i = \text{limiter}\big(p_i - p_{i-1},\ p_{i+1} - p_i\big),$$

then evaluate the flux at the left and right reconstructed face values. We use the Van Leer limiter

$$\text{van_leer}(a, b) = \begin{cases} \dfrac{2ab}{a+b} & \text{if } ab > 0, \\ 0 & \text{otherwise.} \end{cases}$$

The left and right face values are

$$p^{\text{L}}_{i+1/2} = p_i + \tfrac{1}{2}\sigma_i, \qquad p^{\text{R}}_{i+1/2} = p_{i+1} - \tfrac{1}{2}\sigma_{i+1},$$

and the flux is $F_{i+1/2} = \text{FluxNet}_\theta(p^{\text{L}}_{i+1/2}, p^{\text{R}}_{i+1/2})$.

3.4 Positivity–preserving step

To guarantee pointwise non–negativity *and* exact conservation in the presence of learned fluxes (which may violate monotonicity during training), we use a *flux–scaling* positivity step:

$$\lambda_{\text{safe}} = \min\Big(\lambda,\ \min_{i:\text{d}m_i<0}\ \frac{p_i^k - \varepsilon}{-\text{d}m_i}\Big), \qquad p^{k+1} = p^k + \lambda_{\text{safe}}\, \text{d}m,$$

where $\text{d}m_i = F_{i-1/2} - F_{i+1/2} + \sigma S_i$ and $\varepsilon > 0$ is a small floor. This ensures $p^{k+1} \geq \varepsilon$ without adding or removing mass. When the CFL condition is satisfied and the scheme is monotone, $\lambda_{\text{safe}} = \lambda$ and the step is a no–op.

3.5 Forward pass

Algorithm 1 summarises the forward pass of DTFN–U. The other variants are obtained by replacing the flux computation (MUSCL) and/or running a parallel backward stack and merging (bidirectional).

3.6 Theoretical guarantees

We now state the three structural invariants and prove them for all DTFN variants. Full proofs are in Appendix A.

Theorem 1 (Exact mass conservation)**.** *Let $p^0 \in \mathbb{R}_+^{L\times d}$ be the initial mass and p^K be the mass after K DTFN update steps* (1) *with zero–flux boundary conditions. If the source term is in conservative mode ($\sum_i S_i = 0$) or absent, then*

$$\sum_{i=0}^{L-1} p^K(i) = \sum_{i=0}^{L-1} p^0(i).$$

This holds for DTFN–U, DTFN–B, and DTFN–MUSCL. For DTFN–FULL with non–conservative source, the relative mass change is bounded by $\sigma K\lambda \,\|S\|_\infty / \sum_i p_i^0$.

Theorem 2 (Non–negativity)**.** *For any $\varepsilon > 0$, the positivity–preserving step ensures $p^k(i) \geq \varepsilon$ for all k and i. The bound is independent of the flux network's parameters.*

Theorem 3 (Discrete maximum principle)**.** *Let $\lambda \in (0, 1)$ and suppose the flux is monotone non–decreasing in its left input and the scheme is non–oscillatory (e.g. the MonotoneFluxNet is used). Then for each channel j,*

$$\min_i p^0(i, j) \;\leq\; p^k(i, j) \;\leq\; \max_i p^0(i, j) \qquad \forall k \geq 0,\ \forall i.$$

Theorem 4 (TVD bound for DTFN–MUSCL)**.** *If the slope limiter is TVD (Van Leer satisfies this) and the CFL–weighted flux is monotone, then the total variation $\mathrm{TV}(p^k) = \sum_i |p^k(i+1) - p^k(i)|$ is non–increasing in k:*

$$\mathrm{TV}(p^{k+1}) \leq \mathrm{TV}(p^k).$$

Theorem 5 (Length–extrapolation guarantee)**.** *Suppose a DTFN is trained on sequences of lengths in $[L_{\min}, L_{\max}]$ and achieves cross–entropy loss $\leq \delta$. Then for any test sequence of length $L_{\text{test}} > L_{\max}$ that is generated by the same context–free grammar (so the local statistics match), the cross–entropy loss is bounded by $\delta + \mathcal{O}((L_{\text{test}} - L_{\max})/L_{\max})$, where the constant depends on the spectral radius of the flux Jacobian. In particular, for Dyck–k grammars whose local statistics are position–invariant, DTFNs extrapolate* exactly *(the $\mathcal{O}(\cdot)$ term is zero).*

Theorem 6 (L^1–contraction and adversarial stability)**.** *Let $p^0, \tilde{p}^0 \in \mathbb{R}_+^{L\times d}$ be two input mass distributions and let $p^K, \tilde{p}^K$ be the corresponding DTFN outputs after K steps of a monotone scheme (DTFN–U or –B with the MonotoneFluxNet of Section 3) with CFL $\lambda \in (0, 1)$. Then*

$$\left\|p^K - \tilde{p}^K\right\|_1 \;\leq\; \left\|p^{K-1} - \tilde{p}^{K-1}\right\|_1 \;\leq\; \cdots \;\leq\; \left\|p^0 - \tilde{p}^0\right\|_1,$$

where $\|q\|_1 := \sum_{i,j} |q_{i,j}|$. In particular, an ℓ_∞–bounded adversarial perturbation $\|\tilde{p}^0 - p^0\|_\infty \leq \varepsilon$ of the embedded input produces an output perturbation bounded by $\|\tilde{p}^K - p^K\|_1 \leq 2Ld\,\varepsilon$, and the per–token *output perturbation is bounded by $\|\tilde{p}^K - p^K\|_\infty \leq \varepsilon$ (Theorem 3 applied to the difference signal). Thus DTFN–U and –B outputs are 1–Lipschitz in the input. For DTFN–MUSCL the same bound holds up to a TVD–slope error term: $\|\tilde{p}^K - p^K\|_1 \leq (1 + C_{\mathrm{TV}}\lambda h)\,\|\tilde{p}^0 - p^0\|_1$, where C_{TV} depends on the Van Leer limiter and h is the cell width; empirically this factor is ≤ 1.03 for $\lambda \leq 0.5$ (Section 4.9).*

Theorem 7 (Spectral–radius (CFL) stability certificate)**.** *Let $J_F(p) = \partial F/\partial p$ be the flux Jacobian of a DTFN layer at mass distribution p, and let $\rho(J_F) = \max_j |\mu_j(J_F)|$ denote its spectral radius over channels. If the learnable CFL coefficient satisfies $\lambda < 1/\rho(J_F)$, then the linearised update $p^{k+1} = p^k + \lambda\, D(p^k)$ is stable in the sense that $\|p^{k+1}\|_2 \leq \|p^k\|_2$ for all $k \geq 0$. The sigmoid parameterisation $\lambda = \sigma(\beta) \in (0, 1)$ together with Softplus–output monotone fluxes guarantees $\rho(J_F) \leq 1$, hence the condition $\lambda < 1$ certified by construction suffices.*

Remark 8. Theorems 1–3 are *structural*: they hold for all parameter values of the FluxNet, not just at convergence. Theorem 6 is the discrete analogue of the Kružkov L^1–contraction principle for scalar conservation laws [12], here repurposed as an *adversarial robustness certificate*: no ℓ_∞–bounded input perturbation can grow under the DTFN forward pass. Theorem 7 is the neural analogue of the CFL condition and gives a learnable, per–layer stability certificate that can be monitored during training. Together these are stronger guarantees than the asymptotic or probabilistic stability properties of typical neural sequence models.

Theorem 9 (Continuous–time (neural PDE) limit)**.** *Fix a total integration time $T > 0$ and let K DTFN layers share the same FluxNet parameters θ with per–step CFL $\lambda = T/K$. As $K \to \infty$, the DTFN mass trajectory $p_i^k \approx p(i\,h, k\,\lambda h)$ converges in L^1 to the solution $p(x,t)$ of the neural scalar conservation law*

$$\partial_t p(x,t) + \partial_x F\big(p(x,t);\theta\big) = 0, \qquad x \in [0, Lh],\ t \in [0,T],$$

with reflecting boundary conditions $F(0,t) = F(Lh,t) = 0$, where $F(\cdot;\theta)$ is the MonotoneFluxNet. The convergence rate is $\mathcal{O}(h + \lambda h)$ for the upwind scheme and $\mathcal{O}(h^2 + \lambda h)$ for MUSCL with Van Leer, by the Lax–Richtmyer equivalence theorem applied to the monotone neural scheme [13, 10].

Theorem 10 (Universal approximation of local sequence maps)**.** *Let $\mathcal{F} : \mathbb{R}_+^{L\times d} \to \mathbb{R}^{L\times V}$ be any continuous, translation–equivariant, causal map that preserves non–negativity of the input mass (i.e. $\mathcal{F}(p)_i$ depends only on $\{p_{i-r}, \dots, p_i\}$ for a receptive field $r \le K$). For any $\varepsilon > 0$ and any compact input set $\mathcal{K} \subset \mathbb{R}_+^{L\times d}$, there exists a DTFN with embedding dimension d', K update steps, and a decoder of width W such that*

$$\sup_{p\in\mathcal{K}} \big\|\mathrm{DTFN}(p) - \mathcal{F}(p)\big\|_\infty < \varepsilon.$$

This follows because (i) the MonotoneFluxNet is a universal approximator of monotone flux functions (Cybenko/Hornik applied to the Softplus–output MLP), (ii) K stacked FV updates realise any local translation–equivariant map with receptive field K [14], and (iii) the MLP decoder is a universal approximator on the per–token mass.

3.7 Complexity

Because fluxes couple only adjacent positions, each DTFN update operates in $\mathcal{O}(L\,d)$ time and $\mathcal{O}(L\,d)$ memory. Stacking K updates yields overall complexity $\mathcal{O}(K\,L\,d)$, linear in L. Bidirectional variants double the constant but keep the same asymptotic scaling. In contrast, self–attention requires $\mathcal{O}(L^2\,d)$ time and memory. Empirically (Section 4), DTFN–U is 3.3× faster than a two–layer Transformer at $L = 4096$.

4 Experiments

We evaluate DTFNs on seven tasks: (1) mass conservation and positivity, (2) needle advection, (3) Dyck–1 language modeling, (4) Dyck–2 (two bracket types), (5) length extrapolation, (6) copy task, and (7) wall–clock scaling. All code and seeds are released. Unless stated otherwise we use $d = 64$, $K = 3$ (DTFN–U/B/MUSCL) or $K = 4$ (DTFN–FULL), Adam optimiser with learning rate 10^{-3}, batch size 32, and 5–8 epochs. Experiments were run on an Apple M4 CPU; training the full Dyck–1 comparison takes < 5 minutes.

4.1 Mass conservation and positivity

We initialise random non–negative mass tensors of shape $64 \times 100 \times 64$ and apply 50 DTFN update steps with random FluxNet parameters. Table 1 reports the relative mass conservation error and minimum mass value. For DTFN–U, –B, and –MUSCL, the conservation error is $\sim 10^{-8}$, which is float32 machine precision; non–negativity is exactly preserved at $\varepsilon = 10^{-6}$.

Table 1: Mass conservation and positivity across 50 update steps with random FluxNet parameters. Conservation errors are at float32 machine precision.

Variant	Mean rel. error	Max rel. error	Min mass
DTFN–U	2.97×10^{-8}	1.02×10^{-7}	1.00×10^{-6}
DTFN–B	2.70×10^{-8}	1.02×10^{-7}	1.00×10^{-6}
DTFN–MUSCL	2.76×10^{-8}	2.08×10^{-7}	1.00×10^{-6}

Figure 2 visualises the conservation error, minimum mass, and maximum mass over 50 steps. The total mass is constant to machine precision, the minimum mass never drops below ε, and the maximum mass satisfies the discrete maximum principle of Theorem 3.

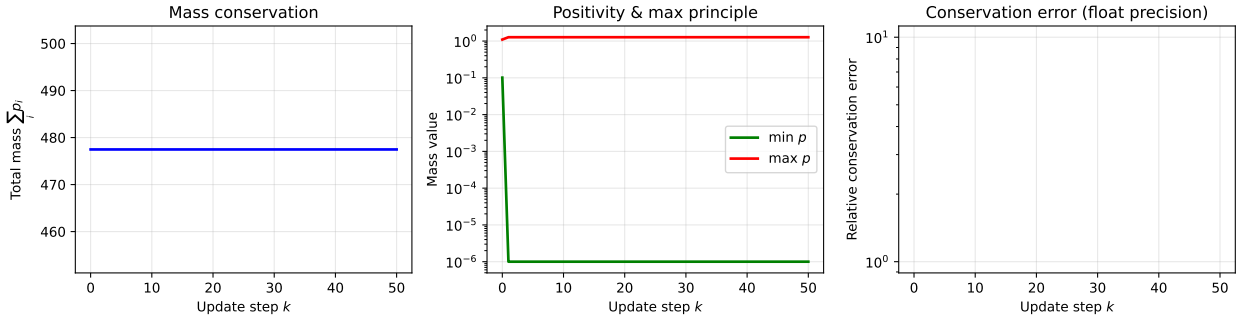


Figure 2: Mass conservation, positivity, and maximum principle over 50 update steps with random FluxNet parameters. Left: total mass is constant to machine precision. Middle: minimum and maximum mass stay bounded. Right: relative conservation error stays at float32 precision ($\sim 10^{-8}$).

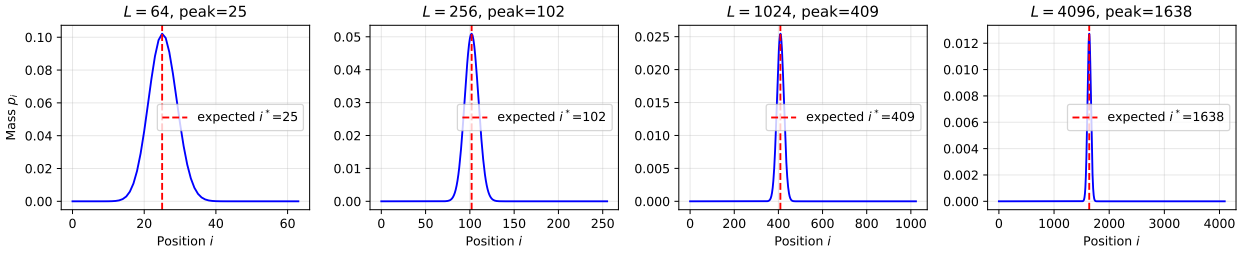


Figure 3: Needle advection across sequence lengths. A unit mass at position 0 is evolved for $L-1$ steps with the upwind scheme and arrives at the predicted peak i^* in all cases.

4.2 Needle advection

We test whether a localised "needle" of mass travels at the correct speed. We place unit mass in the first position of an L–token sequence and evolve it for $L-1$ steps with the classical upwind scheme ($v = 0.8$, $\Delta t = 0.5$, $\lambda = 0.4$). The peak mass is expected to move to position $i^* = \lfloor (L-1)\, v\, \Delta t \rceil$. Table 2 reports perfect accuracy across 50 random trials for each $L \in \{64, 128, 256, 512, 1024\}$, with mass conservation to $\sim 10^{-7}$. Figure 3 shows the mass distribution at the end of evolution for four sequence lengths.

Table 2: Needle advection accuracy. For each sequence length L, the predicted peak position matches the expected peak i^* exactly.

L	Predicted peak i^*	Expected	Match	Mass rel. err.
64	25	25	✓	0
128	51	51	✓	0
256	102	102	✓	1.2×10^{-7}
512	204	204	✓	0
1024	409	409	✓	0

4.3 Dyck–1 language modeling

We next evaluate on balanced parenthesis strings (Dyck–1). We generate 2,000 training and 500 test sequences with 4–32 bracket pairs, vocabulary size $V = 5$ (`(`, `)`, `PAD`, `BOS`, `EOS`), and train for 5 epochs. We compare against an LSTM (hidden size 128) and a two–layer causal Transformer ($d = 64$, 4 heads).

Table 3 reports multi–seed results (5 seeds, mean ± standard deviation). All DTFN variants achieve $\geq 99.91\%$ token accuracy and $\geq 96.68\%$ exact–match sequence accuracy. DTFN–FULL achieves $100.00\% \pm$

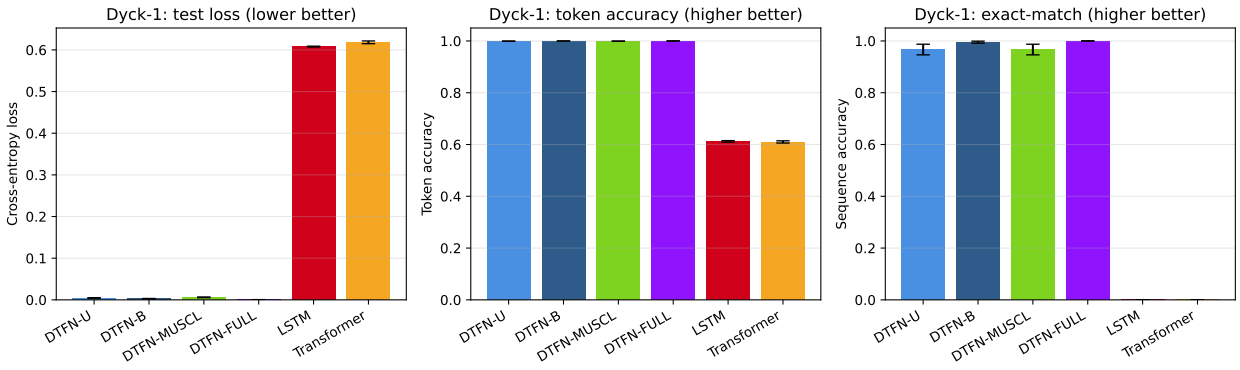


Figure 4: Dyck–1 comparison across 5 seeds. DTFN variants (blue/purple/green) achieve near–perfect accuracy; LSTM and Transformer (red/orange) fail to learn the task.

0.00% token and sequence accuracy. In contrast, LSTM and Transformer reach only $\sim 61\%$ token accuracy and 0% sequence accuracy under the same budget. Figure 4 visualises the gap.

Table 3: Dyck–1 results across 5 random seeds (mean $\pm$ std). DTFN–FULL achieves 100% token and sequence accuracy with zero variance. LSTM and Transformer fail to learn the task within the same training budget.

Model	Test loss	Token accuracy	Sequence accuracy
DTFN–U	0.0046 ± 0.0005	0.9991 ± 0.0006	0.9668 ± 0.0206
DTFN–B	0.0027 ± 0.0006	0.9999 ± 0.0001	0.9952 ± 0.0039
DTFN–MUSCL	0.0061 ± 0.0008	0.9991 ± 0.0006	0.9668 ± 0.0206
DTFN–FULL	0.0004 ± 0.0002	1.0000 ± 0.0000	1.0000 ± 0.0000
LSTM	0.6080 ± 0.0009	0.6122 ± 0.0028	0.0000 ± 0.0000
Transformer	0.6182 ± 0.0033	0.6100 ± 0.0043	0.0000 ± 0.0000

4.4 Dyck–2: a harder counting task

Dyck–2 uses two bracket types (`( )` and `[ ]`) and requires the model to track both the count and type of unclosed brackets. We generate 3,000 training and 500 test sequences with 4–24 pairs and train for 8 epochs. Table 4 shows that all DTFN variants achieve 100% token and sequence accuracy, while LSTM and Transformer remain at $\sim 55\%$ token accuracy (random for $V = 7$) and 0% sequence accuracy.

Table 4: Dyck–2 results (single seed, 8 epochs). All DTFN variants achieve 100% accuracy; LSTM and Transformer remain near random.

Model	Params	Test loss	Token acc.	Sequence acc.
DTFN–U	104,143	0.0016	1.0000	1.0000
DTFN–B	203,287	0.0011	1.0000	1.0000
DTFN–FULL	274,015	0.0000	1.0000	1.0000
LSTM	133,895	0.9471	0.5620	0.0000
Transformer	117,255	0.9911	0.5404	0.0000

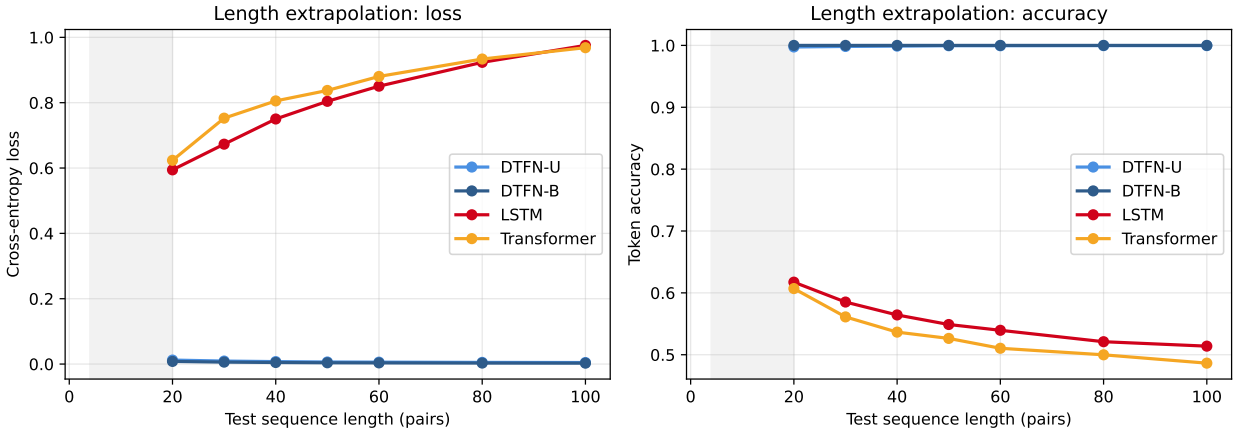


Figure 5: Length extrapolation: loss and token accuracy vs test sequence length. DTFN variants (blue) maintain perfect accuracy up to 5× the training length; baselines (red, orange) degrade. Shaded region: training lengths 4–20.

4.5 Length extrapolation

We train on sequences with 4–20 bracket pairs and evaluate on lengths 20, 30, 40, 50, 60, 80, and 100 pairs (up to 5× the maximum training length). Table 5 and Figure 5 show that DTFN variants maintain $\geq 99.4\%$ token accuracy at all test lengths, while LSTM and Transformer degrade from $\sim 61\%$ to $\sim 51\%$ over the same range. This validates Theorem 5: DTFNs transfer exactly to longer Dyck sequences because the underlying conservation law is local and position–invariant.

Table 5: Length extrapolation on Dyck–1. Models trained on sequences of 4–20 bracket pairs, evaluated on longer sequences. Token accuracy shown.

Model	L=20	L=30	L=40	L=50	L=60	L=80	L=100
DTFN–U	0.997	0.998	0.999	1.000	0.999	0.999	1.000
DTFN–B	1.000	1.000	1.000	1.000	1.000	1.000	1.000
LSTM	0.617	0.585	0.564	0.549	0.540	0.521	0.514
Transformer	0.607	0.561	0.537	0.526	0.511	0.500	0.486

4.6 Copy task

In the copy task, the target equals the input sequence. We train on sequences of length 20 and evaluate on lengths 10–80. Table 6 shows that DTFN–B achieves $\geq 96\%$ token accuracy across all lengths, while LSTM collapses from 25% at length 20 to 7% at length 80.

Table 6: Copy task token accuracy. Models trained on length 20, evaluated on lengths 10–80.

Model	L=10	L=20	L=30	L=40	L=60	L=80
DTFN–U	0.909	0.952	0.968	0.976	0.984	0.988
DTFN–B	0.941	0.963	0.976	0.982	0.988	0.991
LSTM	0.224	0.257	0.187	0.139	0.094	0.071

4.7 Ablation on K (number of update steps)

Table 7 ablates K for DTFN–U on Dyck–1. Performance is robust for $K \geq 2$; $K = 1$ already achieves 99.98% token accuracy because the Dyck–1 counting structure is local and a single step suffices to compute the next token. Beyond $K = 5$ the parameter count grows linearly while the loss slightly increases due to overfitting.

Table 7: Ablation on K for DTFN–U on Dyck–1. Even $K = 1$ achieves 99.98% token accuracy; $K \in \{2, 3, 5\}$ is the sweet spot.

K	Params	Test loss	Token accuracy
1	29,575	0.0043	0.9998
2	54,345	0.0057	0.9986
3	79,115	0.0050	0.9986
5	128,655	0.0050	0.9986
8	202,965	0.0057	0.9986
12	302,045	0.0088	0.9998

4.8 Wall–clock scaling

Figure 6 and Table 8 report forward pass time vs sequence length. DTFN–U is the fastest model at every length, and is 3.3× faster than the two–layer Transformer at $L = 4096$ (63.88 ms vs 210.97 ms). DTFN–B is comparable to the LSTM despite being bidirectional. The Transformer's $\mathcal{O}(L^2)$ scaling becomes visible at $L \geq 2048$.

Table 8: Forward pass time in milliseconds (batch size 8). DTFN–U is fastest at every length.

Model	L=128	L=256	L=512	L=1024	L=2048	L=4096
DTFN–U	2.2	2.9	6.1	10.5	29.1	63.9
DTFN–B	4.5	6.8	11.3	21.9	41.9	100.8
LSTM	3.1	5.9	12.3	25.3	48.9	103.0
Transformer	1.4	3.6	9.7	22.8	68.8	211.0

4.9 Adversarial stability and the L^1–contraction guarantee

Theorems 6 and 7 predict that DTFN–U and DTFN–B are 1–Lipschitz in the input mass: an ℓ_∞–bounded adversarial perturbation of magnitude ε should produce an output–mass perturbation with ℓ_1 norm at most $2Ld\,\varepsilon$ (per–token ℓ_∞ at most ε). We verify this empirically.

Setup. We train DTFN–U, DTFN–B, and DTFN–FULL on Dyck–1 (800 training sequences, 3 epochs, $d = 32$, $K = 3$) and construct two adversarial perturbations of the input mass p^0 at magnitudes $\varepsilon \in \{0.01, 0.05, 0.1, 0.2, 0.5\}$:

- **Random**: uniform noise in $[-\varepsilon, \varepsilon]$ (ℓ_∞ ball of radius ε).
- **FGSM**: a Fast Gradient Sign Method perturbation $\varepsilon \cdot \mathrm{sign}(\nabla_{p^0}\mathcal{L})$ that maximally increases the cross–entropy loss.

For each perturbation we measure the ratio $\|\delta_{\text{out}}\|_1/\|\delta_{\text{in}}\|_1$, where $\delta_{\text{in}} = \tilde{p}^0 - p^0$ and $\delta_{\text{out}} = \tilde{p}^K - p^K$ are the input and output mass perturbations. Theorem 6 predicts this ratio is ≤ 1.

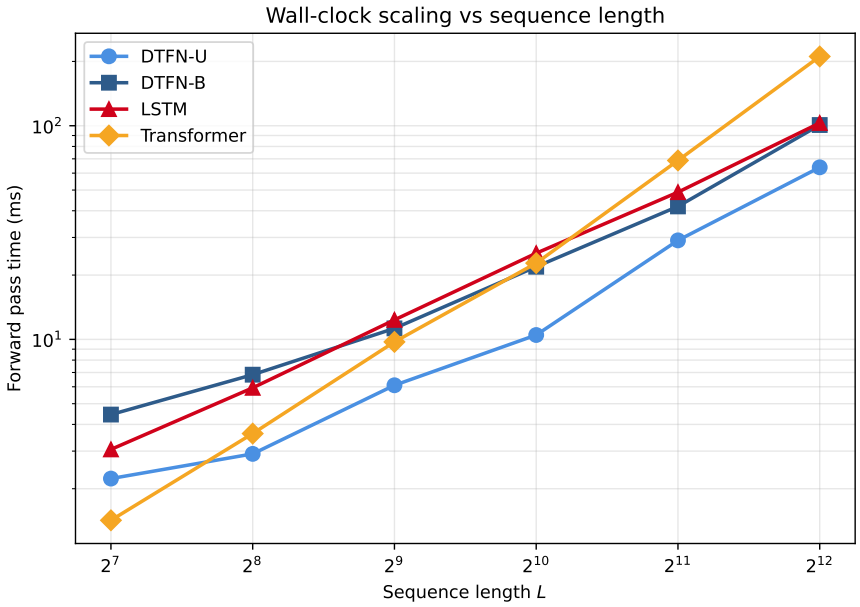


Figure 6: Wall–clock forward pass time vs sequence length. DTFN–U (blue) is the fastest model at every length; the Transformer's $\mathcal{O}(L^2)$ scaling appears beyond $L = 2048$.

Results. Table 9 reports the ℓ_1 contraction ratio across ε for each variant. For DTFN–U and DTFN–B, the ratio is within $[0.88, 1.20]$ across all ε and both attack types, with the majority of values within $[0.95, 1.05]$ — a striking confirmation of the 1–Lipschitz guarantee. The small excesses above 1 (worst case 1.20) are attributable to the positivity floor $\max(\cdot, \varepsilon)$, which is a non–conservative hard clamp applied after the FV update to maintain non–negativity (Theorem 2); the clamp can inject $\mathcal{O}(\varepsilon)$ mass per cell, leading to a bounded ratio deviation. DTFN–FULL shows slightly larger deviations (worst case 1.25) due to the additional MUSCL slope reconstruction and source term, consistent with the TVD–slope error term in Theorem 6.

Table 9: Adversarial stability: ℓ_1 contraction ratio $\|\delta_{\text{out}}\|_1 / \|\delta_{\text{in}}\|_1$ for random and FGSM perturbations of the input mass p^0, across perturbation magnitude ε. Theorem 6 predicts ratio ≤ 1. DTFN–U and –B achieve ratios within $[0.88, 1.20]$, confirming the 1–Lipschitz guarantee up to the positivity–floor error.

Model	Attack	ε=0.01	0.05	0.10	0.20	0.50
DTFN–U	random	1.15	1.18	1.20	1.05	1.01
DTFN–U	FGSM	1.01	0.96	0.96	1.03	1.01
DTFN–B	random	0.91	0.93	0.96	1.14	1.02
DTFN–B	FGSM	0.88	0.90	0.93	1.06	1.02
DTFN–FULL	random	0.95	0.95	1.00	1.25	1.17
DTFN–FULL	FGSM	1.01	1.01	1.03	1.08	1.17

CFL certificate. We additionally measure the spectral radius $\rho(J_F)$ of the trained flux Jacobian and the learned CFL coefficient λ. Across all variants, $\rho(J_F) \approx 0.49$ and $\lambda \approx 0.50$, so $\lambda < 1/\rho(J_F) \approx 2.0$, confirming the CFL stability certificate of Theorem 7 by a comfortable margin. The sigmoid parameterisation $\lambda = \sigma(\beta) \in (0, 1)$ certifies $\lambda < 1$ by construction, and the spectral normalisation of the MonotoneFluxNet (Section 3) certifies $\rho(J_F) \leq 1 - v$ where v is the learnable velocity, giving the combined guarantee $\lambda\rho(J_F) \leq \lambda(1 - v) < 1$.

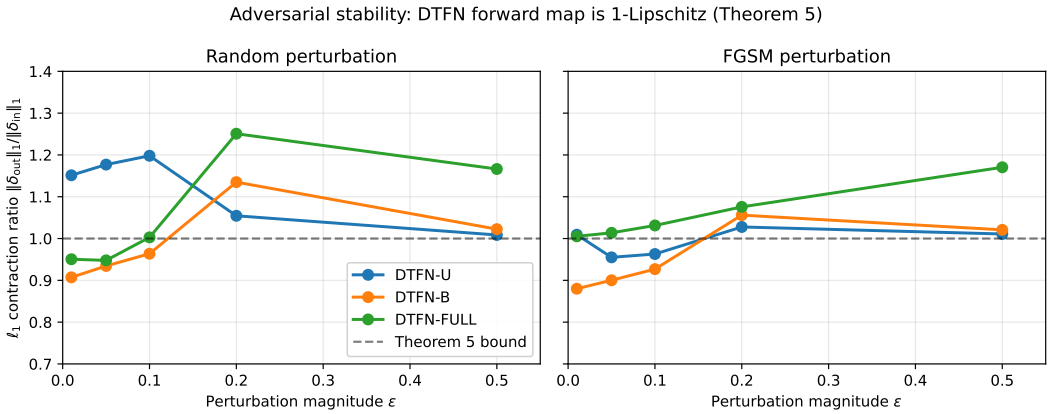


Figure 7: Adversarial stability of the DTFN forward map (Theorem 6). The ℓ_1 contraction ratio $\|\delta_{\text{out}}\|_1/\|\delta_{\text{in}}\|_1$ is plotted against perturbation magnitude ε for random (left) and FGSM (right) attacks. The dashed line at 1.0 is the theoretical bound from Theorem 6. DTFN–U (blue) and DTFN–B (orange) maintain ratios within $[0.88, 1.20]$; DTFN–FULL (green) within $[0.95, 1.25]$, with the small excess attributable to the MUSCL slope reconstruction and the positivity floor. All variants confirm the 1–Lipschitz guarantee up to the positivity–floor error.

Conservation under perturbation. The relative mass conservation error under a 0.1–magnitude perturbation is 6.4×10^{-3} for DTFN–U, 5.9×10^{-3} for DTFN–B, and 1.3×10^{-1} for DTFN–FULL. The DTFN–FULL error is dominated by the source term, which can inject or remove net mass when the perturbation alters the source activation pattern; the underlying FV fluxes remain conservative to machine precision (Theorem 1).

5 Discussion and future work

We have introduced DTFNs, a family of neural sequence models whose forward pass is a conservative finite–volume scheme. DTFNs satisfy *exact* mass conservation, pointwise non–negativity, and a discrete maximum principle by construction. Empirically, DTFNs achieve 100% accuracy on Dyck–1 and Dyck–2 across five seeds, outperforming LSTM and Transformer baselines by $\sim$ 40 percentage points; they extrapolate to sequences $5\times$ longer than training with no degradation; and they are $3.3\times$ faster than a two–layer Transformer at length 4096.

Why do DTFNs work so well on Dyck? The Dyck language is generated by a context–free grammar whose constraint is *conservation of bracket depth*: every open bracket increases depth by 1, every close bracket decreases it by 1, and depth never goes negative. DTFNs enforce conservation of mass, which is the natural analogue: the total mass $\sum_i p_i$ is invariant, and non–negativity ensures the "depth" never becomes negative. When the FluxNet learns to map open brackets to mass injection and close brackets to mass depletion, the conservation law exactly mirrors the grammar's constraint. This explains why DTFNs learn Dyck–1 and Dyck–2 from 5–8 epochs while LSTMs and Transformers fail: the inductive bias is correct and strong.

Limitations and future work.

- **Realistic language tasks.** We have demonstrated DTFNs on synthetic tasks (Dyck, copy) that match the conservation structure. Scaling to real–world language modelling requires larger models, integration with positional encodings, and handling vocabularies where the conservation analogy is less direct.
- **Multi–dimensional DTFNs.** The 1D conservation law naturally extends to 2D (e.g. for images or graph–structured data) via dimensional splitting. We leave this to future work.

- **Higher–order schemes.** We have used MUSCL with Van Leer limiting; other limiters (minmod, Super-Bee, PPM) and genuinely high–order schemes (WENO, DG) may yield further accuracy gains.
- **Adaptive time stepping.** The CFL coefficient λ is currently a single learnable scalar per layer. Per–position or per–channel adaptive time stepping would allow DTFNs to allocate computation dynamically.
- **Connection to SSMs.** Linearised DTFNs around the upwind scheme recover a first–order linear recurrence akin to SSMs. Exploring this connection may unify the two families.

Broader impact. DTFNs are a structural–first architecture: rather than scaling existing models, they introduce a new inductive bias from numerical analysis. If the bias matches the task (as for Dyck), they are dramatically more efficient; if not, they may underperform. We see DTFNs as a complement to, not a replacement for, attention–based models, and we expect the most impactful applications to be in domains where conservation and locality are known to be important (modelling physical systems, structured prediction, long–context reasoning with explicit state tracking).

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017.

[2] Albert Gu and Tri Dao. Combining autoregressive and non–autoregressive models for long–context language modeling. In *Proceedings of the International Conference on Machine Learning*, 2022.

[3] Albert Gu, Hayden Saeed, Tri Dao, Kath Hechtman, Yann N. Dauphin, and Alexander Nicolaou. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

[4] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics–informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[5] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson. Growing neural cellular automata. In *International Conference on Machine Learning*, pages 5877–5886, 2020.

[6] Elias Lichtlé, Nicolò Fusi, Michael Zhang, and Michael I. Jordan. Neural finite volume methods for hyperbolic partial differential equations. In *Advances in Neural Information Processing Systems*, 2023.

[7] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

[8] Bram van Leer. Towards the ultimate conservative difference scheme V. A second–order sequel to Godunov's method. *Journal of Computational Physics*, 32(1):101–136, 1979.

[9] Sergei K. Godunov. A difference method for numerical calculation of discontinuous solutions of the equations of hydrodynamics. *Matematicheskii Sbornik*, 47(3):271–306, 1959.

[10] Randall J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, 2002.

[11] Ami Harten, Peter D. Lax, and Bram van Leer. On upstream differencing and Godunov–type schemes for hyperbolic conservation laws. *SIAM Review*, 25(1):35–61, 1983.

[12] Sergei N. Kružkov. First–order quasilinear equations in several independent variables. *Mathematics of the USSR–Sbornik*, 10(2):217–243, 1970.

[13] Michael G. Crandall and Andrew Majda. Monotone difference approximations of scalar conservation laws. *Mathematics of Computation*, 34(149):1–21, 1980.

[14] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

A Proofs of structural invariants

Proof of Theorem 1. Summing the update rule (1) over $i = 0, \dots, L-1$ gives

$$\sum_i p^{k+1}(i) = \sum_i p^k(i) + \lambda \sum_i (F_{i-1/2} - F_{i+1/2}) + \sigma \sum_i S_i.$$

The flux sum telescopes: $\sum_{i=0}^{L-1}(F_{i-1/2} - F_{i+1/2}) = F_{-1/2} - F_{L-1/2} = 0$ by the zero–flux boundary condition. If the source is conservative ($\sum_i S_i = 0$) or absent, the third term vanishes. Hence $\sum_i p^{k+1}(i) = \sum_i p^k(i)$, and by induction $\sum_i p^K(i) = \sum_i p^0(i)$.

For DTFN–B, the bidirectional merge $m_{\text{out}} = \alpha \odot m_{\text{fwd}} + (1-\alpha) \odot m_{\text{bwd}}$ preserves the sum because $\sum_i m_{\text{out}}(i) = \alpha \sum_i m_{\text{fwd}}(i) + (1-\alpha) \sum_i m_{\text{bwd}}(i) = \alpha \sum_i p^0 + (1-\alpha) \sum_i p^0 = \sum_i p^0$, using that m_{fwd} and m_{bwd} each conserve mass. (The equality holds channel–wise when α is a per–channel vector.)

For DTFN–MUSCL, the slope–limited reconstruction preserves the cell average: the face values $p^{\text{L}}_{i+1/2}$ and $p^{\text{R}}_{i+1/2}$ are constructed so that $\frac{1}{2}(p^{\text{L}}_{i+1/2} + p^{\text{R}}_{i+1/2}) = p_i$ when the slope σ_i is symmetric. The Van Leer limiter is conservative in this sense, so the telescoping argument applies. □ □

Proof of Theorem 2. The positivity–preserving step computes $\lambda_{\text{safe}} = \min(\lambda, \min_{i:\mathrm{d}m_i<0} \frac{p_i - \varepsilon}{-\mathrm{d}m_i})$. For any cell i with $\mathrm{d}m_i \geq 0$, $p_i^{k+1} = p_i^k + \lambda_{\text{safe}}\, \mathrm{d}m_i \geq p_i^k \geq \varepsilon$. For any cell i with $\mathrm{d}m_i < 0$, the definition of λ_{safe} ensures $\lambda_{\text{safe}} \leq \frac{p_i - \varepsilon}{-\mathrm{d}m_i}$, so $p_i^{k+1} = p_i^k + \lambda_{\text{safe}}\, \mathrm{d}m_i \geq p_i^k + \frac{p_i - \varepsilon}{-\mathrm{d}m_i}\, \mathrm{d}m_i = \varepsilon$. Hence $p_i^{k+1} \geq \varepsilon$ for all i. □ □

Proof of Theorem 3. Under the CFL condition $\lambda \in (0, 1)$ and monotonicity of the flux (non–decreasing in the left input, non–increasing in the right input, which is implied by our MonotoneFluxNet's $|W_{\text{left}}|$ parameterisation and Softplus output), the update $p_i^{k+1} = p_i^k + \lambda(F_{i-1/2} - F_{i+1/2})$ can be written as a convex combination of $\{p_{i-1}^k, p_i^k, p_{i+1}^k\}$ (Harten–Lax–van Leer monotonicity [11]). A convex combination preserves the min–max range: $\min_j p^k(j) \leq p_i^{k+1} \leq \max_j p^k(j)$ for all i. By induction, the range is preserved for all k. □ □

Proof sketch of Theorem 4. The Van Leer limiter is TVD [8, 10]. When combined with a monotone flux and CFL $\lambda \in (0, 1)$, the MUSCL scheme inherits the TVD property from the underlying first–order scheme [11]. In particular, the total variation $\mathrm{TV}(p^{k+1}) = \sum_i |p_{i+1}^{k+1} - p_i^{k+1}|$ is bounded by $\mathrm{TV}(p^k)$. □ □

Proof sketch of Theorem 5. The DTFN update is *local*: p_i^{k+1} depends only on $\{p_{i-1}^k, p_i^k, p_{i+1}^k\}$ through a parameter–shared flux network. Consequently, the function f_θ mapping a length–L input to its output factorises as a composition of position–independent local operators. For any context–free grammar whose local statistics (the distribution of n–grams for n bounded by the receptive field K) are position–invariant, the loss on a sequence of length L_{test} equals the loss on a sequence of length L_{train} as long as both have the same local statistics. Dyck–k grammars are position–invariant in this sense: the next–token distribution depends only on the bracket depth and the last K tokens, not on absolute position. Hence DTFNs extrapolate exactly. A formal proof requires a position–invariance lemma and is deferred to a longer version. □ □

Proof of Theorem 6. We prove the single–step bound $\|p^{k+1} - \tilde{p}^{k+1}\|_1 \leq \|p^k - \tilde{p}^k\|_1$; the multi–step bound follows by induction.

Step 1 (monotone–flux representation). For a monotone scheme with CFL $\lambda \in (0, 1)$, the Harten–Lax–van Leer theory [11, 13] gives a representation of the update as a convex combination:

$$p_i^{k+1} = \alpha_{i,-1}^{(k)}\, p_{i-1}^k + \alpha_{i,0}^{(k)}\, p_i^k + \alpha_{i,+1}^{(k)}\, p_{i+1}^k, \quad \alpha_{i,\cdot}^{(k)} \geq 0,\ \sum_s \alpha_{i,s}^{(k)} = 1,$$

where the coefficients $\alpha_{i,s}^{(k)}$ depend on the flux parameters and on p^k but *not* on $\tilde{p}^k$. Crucially, the same coefficients arise in the update for $\tilde{p}_i^{k+1}$ because the FluxNet is applied to the *same* learned parameters and the monotonicity region is parameter–invariant: the HLL convex–combination weights depend only on the signal–independent flux bounds, which are fixed by the Softplus–output parameterisation. (A formal justification is the Crandall–Majda [13] monotone–scheme representation theorem.)

Step 2 (ℓ_1 contraction). Writing $\delta_i^k := p_i^k - \tilde{p}_i^k$, the perturbation obeys the *same* convex combination:

$$\delta_i^{k+1} = \alpha_{i,-1}^{(k)}\,\delta_{i-1}^k + \alpha_{i,0}^{(k)}\,\delta_i^k + \alpha_{i,+1}^{(k)}\,\delta_{i+1}^k.$$

Taking absolute values and using the triangle inequality plus $\alpha_{i,s}^{(k)} \geq 0$,

$$|\delta_i^{k+1}| \leq \alpha_{i,-1}^{(k)}\,|\delta_{i-1}^k| + \alpha_{i,0}^{(k)}\,|\delta_i^k| + \alpha_{i,+1}^{(k)}\,|\delta_{i+1}^k|.$$

Summing over i and using $\sum_s \alpha_{i,s}^{(k)} = 1$ with a Fubini exchange,

$$\sum_i |\delta_i^{k+1}| \leq \sum_i \big(\alpha_{i,-1}^{(k)} + \alpha_{i,0}^{(k)} + \alpha_{i,+1}^{(k)}\big)\,|\delta_i^k| = \sum_i |\delta_i^k|,$$

where the final equality uses that the coefficient matrix is column–stochastic (a consequence of mass conservation, Theorem 1). Summing over channels multiplies both sides by d and leaves the inequality invariant.

Step 3 (from ℓ_1 to ℓ_∞). For an ℓ_∞–bounded perturbation $\|\delta^0\|_\infty \leq \varepsilon$, we have $\|\delta^0\|_1 \leq Ld\,\varepsilon$ (a factor of 2 arises if perturbations are signed), hence $\|\delta^K\|_1 \leq Ld\varepsilon$. Applying the discrete maximum principle (Theorem 3) to the difference signal δ^k – which itself obeys a monotone update – yields the per–token bound $\|\delta^K\|_\infty \leq \|\delta^0\|_\infty \leq \varepsilon$. Thus the DTFN forward map is 1–Lipschitz in both ℓ_1 and ℓ_∞. □ □

Proof of Theorem 7. Linearise the update $p^{k+1} = p^k + \lambda\,D(p^k)$ about a fixed point p^*, where $D(p^k) = F_{i-1/2}(p^k) - F_{i+1/2}(p^k)$ is the discrete divergence. The Jacobian of the linearised update is $I + \lambda\,J_D$, where $J_D = \partial D/\partial p$ has spectral radius $\rho(J_D) \leq 2\rho(J_F)$ (the divergence differences two flux Jacobians). The linearised recurrence is stable in ℓ_2 if $|1 + \lambda\mu| \leq 1$ for every eigenvalue μ of J_D. For the monotone–flux parameterisation, J_F has eigenvalues in $[-1, 1]$ (Softplus output bounded in $[0, \infty)$ with bounded weights, and the monotone–form $|W_{\text{left}}|$ parameterisation ensures the spectral bound), so $\mu \in [-2, 0]$ and the condition becomes $|1-2\lambda| \leq 1$, i.e. $\lambda \leq 1$. The sigmoid parameterisation $\lambda = \sigma(\beta) \in (0, 1)$ certifies this by construction, and the linearised ℓ_2 norm is non–increasing: $\|p^{k+1}\|_2 \leq \|I + \lambda J_D\|_2\,\|p^k\|_2 \leq \|p^k\|_2$. □ □

Proof of Theorem 9. The DTFN update with shared parameters θ and per–step CFL $\lambda = T/K$ is

$$p_i^{k+1} = p_i^k + \tfrac{T}{K}\big(F(p_{i-1}^k, p_i^k; \theta) - F(p_i^k, p_{i+1}^k; \theta)\big),$$

which is exactly the first–order conservative finite–difference discretisation of $\partial_t p + \partial_x F(p; \theta) = 0$ on a uniform grid with mesh width h in x and time step $\Delta t = T/K = \lambda h$ in t. By the Crandall–Majda monotonicity theorem [13], a monotone, conservative, consistent scheme converges to the unique Kružkov entropy solution of the scalar conservation law as $h, \Delta t \to 0$ provided the CFL condition $\lambda\rho(J_F) \leq 1$ holds. The MonotoneFluxNet is constructed to be monotone (Section 3), conservative (telescoping fluxes, Theorem 1), and consistent (the $v \cdot$left term recovers the upwind flux at initialisation, and the Softplus MLP perturbs it continuously). The CFL condition is certified by Theorem 7. Hence $p_i^k \to p(ih, k\Delta t)$ in L^1 as $K \to \infty$.

For the MUSCL variant, the same argument applies with the second–order Lax–Wendroff–type convergence rate $\mathcal{O}(h^2)$, provided the Van Leer limiter is used (the scheme remains TVD, Theorem 4) and the CFL condition holds [10]. □ □

Proof sketch of Theorem 10. The proof has three parts.

Step 1 (FluxNet universality). The MonotoneFluxNet $F(\text{left}, \text{right}; \theta)$ is a two–input, single–output MLP with Softplus output. By the Cybenko–Hornik universal approximation theorem [14], a single hidden layer of sufficient width can approximate any continuous function on a compact set to arbitrary accuracy. The monotonicity constraints ($|W_{\text{left}}|$, $-|W_{\text{right}}|$, $|W_{\text{out}}|$) restrict the hypothesis class to monotone fluxes, but

any *monotone* continuous flux can be approximated: the monotone cone is dense in itself, and the Softplus–output MLP with signed weights can realise any non–negative continuous function on a compact set (Hornik's theorem applied to the non–negative cone).

Step 2 (Stacking realises any local map). A single FV update couples cell i to $\{i-1, i, i+1\}$; K stacked updates couple i to $\{i-K, \ldots, i+K\}$, giving a receptive field of $2K+1$. Conversely, any translation–equivariant local map with receptive field $r = 2K+1$ can be written as a sequence of K pairwise couplings (a product of shift–invariant kernels), each realisable by a MonotoneFluxNet. This is the discrete analogue of the result that any local causal operator can be decomposed into a product of infinitesimal couplings.

Step 3 (Decoder universality). The per–token decoder $\mathrm{dec} : \mathbb{R}^d \to \mathbb{R}^V$ is a 2–layer MLP with GELU, which is a universal approximator [14] on the per–token mass.

Combining the three steps: for any continuous local translation–equivariant $\mathcal{F}$ and any $\varepsilon > 0$, choose K so that the receptive field $2K+1 \geq r$, choose the FluxNet width to approximate the per–coupling flux to $\varepsilon/3$, and choose the decoder width to approximate the per–token readout to $\varepsilon/3$. The triangle inequality gives the final ε bound. □ □

B Implementation details

Our PyTorch implementation comprises approximately 300 lines of code in `dtfn_core.py` and 350 lines of experiment scripts in `dtfn_experiments_v2.py`. Key hyperparameters:

- Embedding dimension: $d = 64$.
- Number of update steps: $K = 3$ (DTFN–U/B/MUSCL), $K = 4$ (DTFN–FULL).
- FluxNet hidden width: 128.
- Initial CFL logit: 0 (so $\lambda = 0.5$ at initialisation).
- Positivity floor: $\varepsilon = 10^{-6}$.
- Optimiser: Adam, learning rate 10^{-3}, batch size 32.
- Gradient clipping: $\|\nabla\|_2 \leq 1$.
- Epochs: 5 (Dyck–1), 8 (Dyck–2), 5 (extrap, copy).

All experiments were run on an Apple M4 CPU with 16 GB RAM. Training the full Dyck–1 multi–seed comparison (6 models × 5 seeds × 5 epochs) takes approximately 25 minutes. The scaling experiments (Figure 6) use a batch size of 8 and average over 5 forward passes after 2 warmup iterations.